

Mapa detalhado de evidências — NPU Exynos 2400

Projeto: Xclipse Open Project

Ramo: plataforma NPU/NNAPI/ENN do Exynos 2400

Dispositivo de referência: Samsung SM-S721B

Data da coleta: 17–18/09/2026

Escopo: documentar a plataforma observada, os caminhos de acesso, as provas de execução e os bloqueios. Este documento não descreve o XLIA nem implementa uma integração de aplicativo.

1. Leitura correta do resultado

A coleta não produziu uma única conclusão binária do tipo “a NPU está aberta” ou “a NPU está fechada”. Ela revelou **duas superfícies diferentes**.

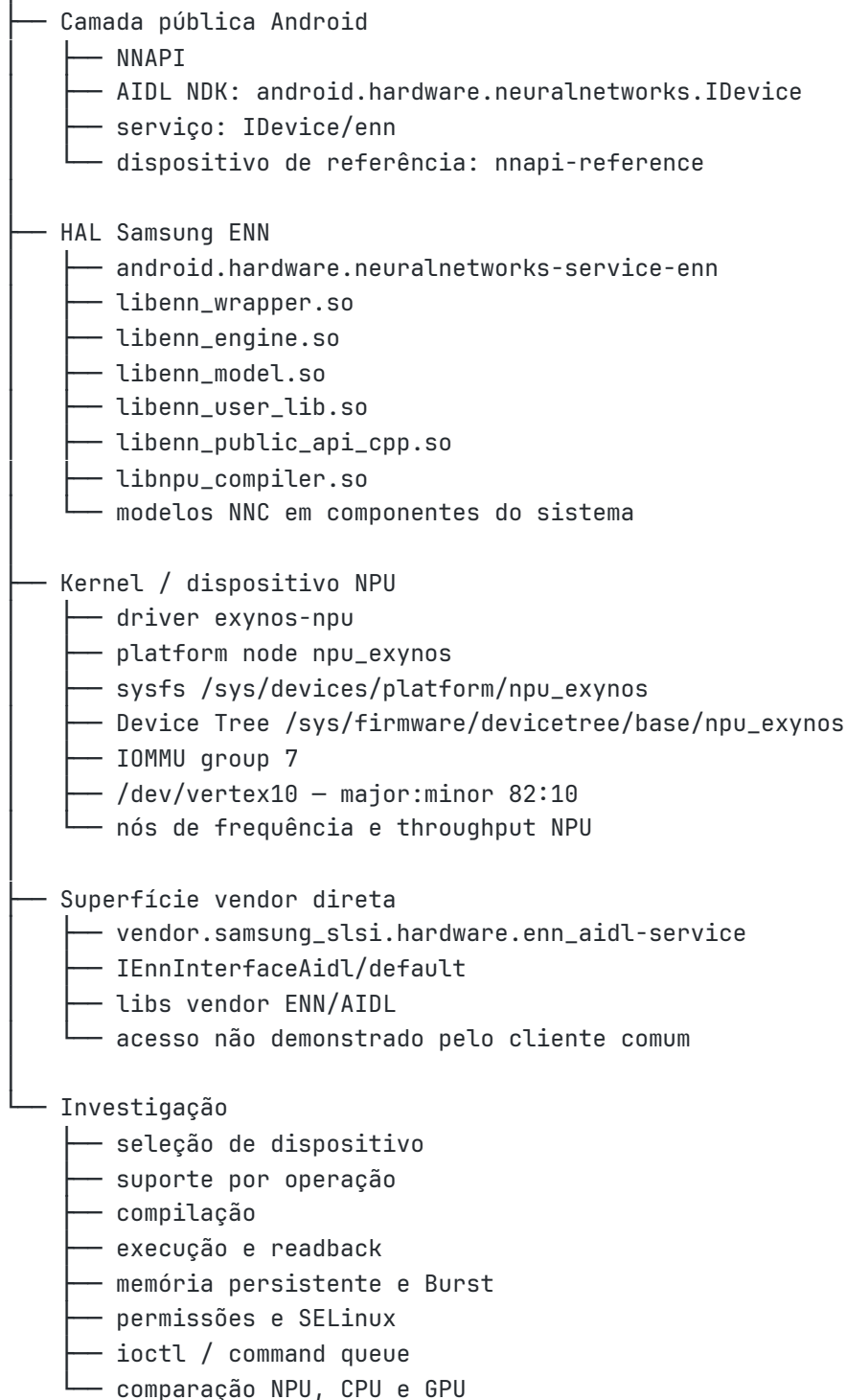
A primeira é a superfície pública NNAPI. Um processo sem root descobriu `android.hardware.neuralnetworks.IDevice/enn`, enumerou o dispositivo ENN, compilou um grafo e executou `SOFTMAX` com readback numérico correto. Essa é uma rota confirmada.

A segunda é a superfície vendor direta. O serviço `vendor.samsung_slsi.hardware.enn_aidl.IEnnInterfaceAidl/default` não foi encontrado pelos probes realizados. O endpoint `/dev/vertex10` exige permissões que o UID comum não possui, e os ioctls tentados com root não produziram uma operação funcional. Essa rota permanece bloqueada ou incompleta.

Portanto, o mapa deve mostrar simultaneamente uma **estrada pública funcional** e uma **estrada vendor direta com bloqueios**.

2. Mapa de arquitetura observada

Exynos 2400 / Android



3. Inventário dos experimentos

Item	Tema	Resultado	Classe
01	Serviços Binder e bibliotecas	IDevice/enn encontrado; bibliotecas ENN relacionadas enumeradas	Probe confirmado
02	Interfaces Binder, AIDL, HIDL e VINTF	Manifesto IDevice/enn , HAL e arquivos de compatibilidade encontrados	Evidência estrutural confirmada
03	Permissões e SELinux	HAL no domínio hal_neuralnetworks_service_enn_default ; libs com rótulos vendor/HAL	Limite de segurança confirmado
03b	Complemento SELinux	Arquivos de policy presentes; nenhuma regra pública suficiente para liberar o endpoint direto	Probe parcial
04	Dispositivos e endpoints	/dev/vertex10 identificado como dispositivo NPU	Probe confirmado
04b	Contexto do dispositivo	vendor_npu_device:s0 , system:system , major:minor 82:10	Permissão observada
08	Rastreamento de aplicação real	Processos de sistema e referências a /sys/ ... /npu_exynos ; serviço vendor direto não apareceu conectado	Observação parcial
09	UID normal versus root	Ambos falharam ao localizar o AIDL vendor direto	Resultado negativo específico
09b	Polling do AIDL vendor	Pollings repetidos retornaram nulo ou foram prejudicados por binário ausente/permissão de execução	Resultado misto; não generalizar
09c	Diagnóstico de coleta	Uma rodada falhou porque o binário não estava presente	Falha de instrumentação
10	Binder NNAPI HAL	Binder remoto IDevice/enn , ping STATUS_OK sem root	Smoke test confirmado
11	Suporte a SOFTMAX	enn reportou SOFTMAX suportado	Suporte por operação confirmado
12	Execução SOFTMAX	Compilação, execução e readback corretos sem root	Execução confirmada
13	BATCH_MATMUL inicial	Modelo inválido	Falha de montagem do

Item	Tema	Resultado	Classe
			teste
13b	BATCH_MATMUL corrigido	Operação não suportada; compilação ENN terminou com status 4	Limite do driver/compiler confirmado
14	Benchmark inicial	Modelo inválido	Falha de montagem do teste
14b	Benchmark corrigido	CPU executou; ENN não compilou	Comparação incompleta
15	Abertura de vertex10	UID comum recebeu <code>Permission denied</code>	Bloqueio rootless confirmado
16	SELinux permissive e ioctl	Root abriu após <code>setenforce 0</code> ; ioctls retornaram <code>EINVAL / EFAULT</code>	Probe privilegiado inconclusivo
17	Descoberta de ioctl	Não houve símbolo público claro nem comando funcional identificado	Investigação aberta
17b	Sysfs do vertex	<code>/dev/vertex10</code> e <code>vertex11</code> aparecem; ligação com <code>/sys</code> confirmada	Topologia confirmada
18	Sysfs e Device Tree	Driver, IOMMU, governors, DVFS, command data e nós NPU expostos	Arquitetura estrutural confirmada

4. Serviços e interfaces

4.1 Serviço público confirmado

A listagem Binder encontrou:

```
android.hardware.neuralnetworks.IDevice/enn
```

O teste sem root obteve um Binder não nulo, confirmou `isRemote() = true` e recebeu `AIBinder_ping() = 0`, interpretado como `STATUS_OK`. O retorno `AIBinder_prepareTransaction() = -38` não invalida a descoberta; ele mostra apenas que aquela preparação específica não era suportada pelo caminho usado no probe.

4.2 VINTF e HAL

Os arquivos observados incluem:

```
/vendor/etc/vintf/manifest/android.hardware.neuralnetworks-service-enn.xml
/vendor/bin/hw/android.hardware.neuralnetworks-service-enn
/vendor/etc/init/android.hardware.neuralnetworks-service-enn.rc
/vendor/etc/init/enn-lazy.rc
/vendor/lib64/android.hardware.neuralnetworks@1.0.so
/vendor/lib64/android.hardware.neuralnetworks@1.1.so
/vendor/lib64/android.hardware.neuralnetworks@1.2.so
/vendor/lib64/android.hardware.neuralnetworks@1.3.so
```

O manifesto declara `IDevice/enn`. A propriedade observada indicou o serviço ativo:

```
init.svc.neuralnetworks_hal_service_enn = running
```

As matrizes de compatibilidade do sistema também mencionam `android.hardware.neuralnetworks` e `vendor.samsung_slsi.hardware.enn`.

4.3 Superfície vendor direta

O sistema contém o binário:

```
/vendor/bin/hw/vendor.samsung_slsi.hardware.enn_aidl-service
```

Também existe a biblioteca:

```
/vendor/lib64/vendor.samsung_slsi.hardware.enn_aidl-V1-ndk.so
```

Mesmo assim, o nome pesquisado abaixo retornou nulo nas rodadas de `checkService`:

```
vendor.samsung_slsi.hardware.enn_aidl.IEnnInterfaceAidl/default
```

A presença do binário e da biblioteca prova que a superfície existe no firmware. Não prova que ela esteja registrada, disponível para qualquer cliente ou autorizada para um aplicativo comum.

5. Bibliotecas e componentes observados

A coleta enumerou componentes de sistema e vendor associados ao ENN/NPU:

```
libenn_wrapper_system.so
libneural.snap.samsung.so
libneuralnetworks_packageinfo.so
libenn_common_utils.so
libenn_cpu_operators.so
libenn_engine.so
libenn_engine_lib.so
libenn_gc_vendor.so
libenn_klm_vendor.so
libenn_model.so
libenn_public_api_cpp.so
libenn_public_api_cpp_lib.so
libenn_seva_vendor.so
libenn_user.samsung_slsi.so
libenn_user_driver_cpu.so
libenn_user_driver_gpu.so
libenn_user_driver_gpu_lib.so
libenn_user_driver_unified.so
libenn_user_lib.so
libenn_wrapper.so
libnpu_compiler.so
libsait_npu_compiler.so
```

A coleta também observou `KnoxNeuralNetworkRuntime.apk`, o serviço HAL e arquivos `.nnc` associados a processamento de câmera. Esses elementos indicam que o sistema utiliza formatos e componentes específicos para modelos neurais. Eles não devem ser copiados para um aplicativo nem redistribuídos como parte do XOP.

6. SELinux, usuários e permissões

O processo HAL foi observado no contexto:

```
u:r:hal_neuralnetworks_service_enn_default:s0
```

O executável recebeu o rótulo:

```
u:object_r:hal_neuralnetworks_service_enn_default_exec:s0
```

O serviço vendor AIDL recebeu:

```
u:object_r:hal_enn_default_exec:s0
```

O endpoint direto recebeu:

```
u:object_r:vendor_npu_device:s0
```

O arquivo apareceu com dono e modo semelhantes a:

```
crw-r--r-- system system 82,10 /dev/vertex10
```

Apesar da aparência permissiva do modo Unix, o cliente comum recebeu `errno=13`, `Permission denied`. Isso demonstra que modo Unix, domínio SELinux e política efetiva precisam ser analisados juntos.

O UID comum observado foi `10415`, no domínio `u:r:untrusted_app_27:s0`. A comparação root utilizou UID 0 no domínio `u:r:ksu:s0`. A diferença de contexto é material: root não reproduz o ambiente de um aplicativo Android normal.

7. Endpoints, sysfs e topologia

A coleta estabeleceu a correspondência:

```
/dev/vertex10
  ↓ major:minor 82:10
/sys/class/vision4linux/vertex10
  ↓
/sys/devices/platform/npu_exynos/vision4linux/vertex10
  ↓
/sys/devices/platform/npu_exynos
  ↓
/sys/bus/platform/drivers/exynos-npu
```

O nó `vertex10` aponta para `npu_exynos`. A plataforma também possui `vertex11` em referências de descoberta, embora a investigação principal tenha usado `vertex10`.

O sysfs de `npu_exynos` expôs, entre outros, os seguintes grupos:

- `driver`, `subsystem`, `modalias` e `uevent`;
- `iommu` e `iommu_group`;
- `power` e `wakeup`;
- `qos_freq`;
- `afm_irp`, `afm_mode`, `afm_onoff`, `afm_restore_msec`, `afm_tdc_threshold` e `afm_tdt`;
- `log_level`, `npu_err_in_dmesg`, `suspend_resume_test` e `version`;
- `vision4linux`;

- nós de frequência e throughput relacionados ao NPU.

A existência de atributos de DVFS, QoS, AFM, wakeup e erro permite construir uma investigação térmica e de frequência. Ela não significa que um aplicativo comum possa escrever nesses atributos.

8. Device Tree observado

O nó foi identificado como:

```
name = npu_exynos
compatible = samsung,exynos-npu
OF_FULLNAME = /npu_exynos
DRIVER = exynos-npu
```

O Device Tree inclui referências a:

- `iommus` ;
- `samsung,iommu-group` ;
- `sysmmu,best-fit` ;
- `dma-coherent` e `dma-window` ;
- `interrupts` ;
- `samsung,npumem-address` e `samsung,npumem-names` ;
- `samsung,npurmem-address` ;
- `samsung,npunode-names` , `samsung,npunode-num` e `samsung,npunode-freq` ;
- `samsung,npusys-corenum` ;
- `samsung,npusched-dvfs` , `samsung,npusched-names` , `samsung,npusched-min-active-cores` e `samsung,npusched-afmlimit` ;
- `samsung,npugovernor-npufreq` , `samsung,npugovernor-intfreq` e `samsung,npugovernor-miffreq` ;
- `samsung,npudvfs-open-clock` , `samsung,npudvfs-close-clock` , `samsung,npudvfs-open-dvfs` , `samsung,npudvfs-close-dvfs` e tabelas DVFS;
- `samsung,npuinter-isr-cpu-affinity` ;
- dezenas de propriedades `samsung,npucmd-*` para controle de clock, DSP, DNC, GNPu, NPU memory e STM;
- `samsung,imgloader-s2mpu-support` ;
- `vertex_name` .

Esses nomes revelam a riqueza da interface interna do driver. Eles não são uma API pública. A documentação os correlaciona com o comportamento observável por NNAPI sem tratar registradores ou atributos de produção como interface de aplicativo.

9. Execução NNAPI/ENN confirmada sem root

9.1 Descoberta

O teste encontrou dois dispositivos:

```
[0] nome=enn tipo=4  
[1] nome=nnapi-reference tipo=2
```

O primeiro é o dispositivo vendor ENN; o segundo é a referência CPU. O teste selecionou explicitamente `enn`.

9.2 Suporte por operação

Para `SOFTMAX`, o modelo terminou com status zero e `getSupportedOperationsForDevices` retornou sucesso. O dispositivo `enn` foi reportado como suportando a operação.

9.3 Compilação e execução

A sequência confirmada foi:

```
ModelFinish = 0  
CompilationCreateForDevices status = 0  
CompilationFinish status = 0  
Execution_compute status = 0
```

O resultado foi:

```
Output do driver: [0.032059, 0.087144, 0.236883, 0.643914]  
Softmax esperado: [0.032059, 0.087144, 0.236883, 0.643914]  
Diferença máxima: 0.000000
```

Como houve compilação, execução e comparação numérica, esta prova é mais forte que enumeração ou simples criação de um objeto NNAPI.

10. Matriz de operações e workloads

Workload	ENN	CPU/reference	Interpretação
<code>SOFTMAX</code> pequeno	Compilou e executou; diferença máxima 0,000000	Resultado esperado igual	Execução ENN confirmada sem root
<code>SOFTMAX</code> de 1024 elementos	Aproximadamente 1,0264 ms	Aproximadamente 0,0104 ms	Funcionalidade não implica vantagem de desempenho
Soma elementar	Resultado <code>[11, 22, 33, 44]</code> em teste anterior	Resultado correto	Execução mínima observada
<code>FULLY_CONNECTED</code> INT8, quatro camadas	Aproximadamente 1,1148 ms; checksum igual	Aproximadamente 6,2407 ms	Melhor evidência de vantagem ENN nos grafos testados
<code>FULLY_CONNECTED</code> FP	Aproximadamente 1,2092 ms	Aproximadamente 0,1332 ms	CPU mais rápida no workload observado
<code>BATCH_MATMUL</code> inicial	Modelo inválido	Não comparável	Falha de montagem do modelo
<code>BATCH_MATMUL</code> corrigido	<code>getSupportedOperationsForDevices</code> : não suportado; compilação status 4	Não usado como comparação equivalente	Bloqueio específico do ENN
Benchmark corrigido	<code>create=0</code> , <code>finish=4</code>	20 execuções em 207,13 ms; 10,356 ms/execução	Benchmark NPU incompleto porque ENN não compilou

Os números não devem ser transformados em uma afirmação geral de que a NPU é mais rápida que a CPU. Eles são específicos dos grafos, formas, tipos, tamanhos e estado do dispositivo.

11. Investigação do endpoint `vertex10`

O cliente sem root tentou:

```
open(O_RDONLY)
open(O_RDWR)
```

Ambas as tentativas retornaram `errno=13`, `Permission denied`.

Com root e `setenforce 0` temporário, o probe conseguiu abrir o endpoint e mapear memória:

```
open(O_RDONLY) fd=3
mmap() = sucesso
```

Entretanto, os comandos testados retornaram:

```
ioctl(cmd=0) = EINVAL
ioctl(cmd=1) = EINVAL
ioctl(cmd=2) = EFAULT
ioctl(cmd=3) = EINVAL
ioctl(cmd=4) = EINVAL
ioctl(cmd=5) = EINVAL
```

O retorno mostra que abrir e mapear não equivale a conhecer o protocolo do driver. A investigação de strings encontrou referências como `REG_BASE_ADDR_CMDQ_CORE`, `runCMDQGenerator`, `runISAGenerator`, `NCP_BINARY`, `NCP Version` e `ioctl`, mas não forneceu uma tabela pública de comandos utilizável.

A ausência de headers em `/system` e `/vendor` para `vertex` ou `NPU_IOCTL` também impede afirmar uma ABI estável.

12. Serviço vendor: por que a rota falhou

O teste `checkService` procurou repetidamente:

```
vendor.samsung_slsi.hardware.enn_aidl.IEnnInterfaceAidl/default
```

As rodadas retornaram nulo. Houve também polling de 30 segundos enquanto a câmera estava aberta e ativa, sem obter um serviço funcional. Algumas rodadas adicionais foram inválidas por falhas de instrumentação: binário ausente, execução em local sem permissão ou tentativa de executar arquivo em armazenamento externo.

Essas falhas precisam ser separadas em duas categorias:

1. **Falha real observada:** o serviço não foi encontrado pelo cliente nas rodadas válidas.
2. **Falha de coleta:** algumas rodadas não chegaram a executar o probe correto.

A conclusão segura é negativa apenas para a rota válida testada naquele estado do aparelho. Não é correto afirmar que o serviço vendor não existe, pois o binário e a biblioteca estão presentes no firmware.

13. Caminhos verdes, bloqueios e áreas abertas

Caminhos confirmados

- NNAPI pública com dispositivo `enn`.
- Processo sem root obtendo Binder remoto.
- Consulta de suporte de operação.
- Compilação de `SOFTMAX` pequeno.
- Execução com readback correto.
- Device Tree e sysfs do driver `exynos-npu`.
- Identificação de `/dev/vertex10` e major:minor `82:10`.

Bloqueios confirmados

- Acesso comum direto a `/dev/vertex10`.
- Suporte a `BATCH_MATMUL` no formato testado.
- Serviço AIDL vendor direto não descoberto.
- ABI de ioctl não identificada.
- Benchmark ENN versus CPU incompleto para o modelo corrigido.
- Algumas rodadas de polling invalidadas por problemas de execução do binário.

Áreas de investigação

- `ANeuralNetworksBurst` e redução do overhead de submissão.
- Memória compartilhada e lifetime de operandos.
- Reuso de compilação e execução persistente.
- Particionamento por operação com fallback explícito.
- Formas alternativas de matriz e tipos suportados.
- Correlacionamento de sysfs, frequência, erro e tempo por execução.
- Registro condicional do serviço vendor em estados específicos do sistema.
- Relação entre ENN, `libnpu_compiler`, NCP e command queue, sem acesso direto destrutivo.

14. O que não está provado

A coleta não prova que:

- todo operador NNAPI seja executado na NPU;
- toda execução `enn` seja mais rápida que CPU;

- o dispositivo `enn` não use fallback interno para uma operação específica;
- um decoder transformer completo seja aceito;
- o endpoint `vertex10` seja uma ABI pública;
- o serviço vendor AIDL possa ser usado por qualquer aplicativo;
- a presença de modelos `.nnc` permita reutilização por terceiros;
- a escrita em sysfs ou o uso de root transforme a interface interna em API estável.

15. Arquitetura consolidada observada

15.1. Camadas de software

A arquitetura observada começa na aplicação ou cliente NNAPI. O cliente não conversa diretamente com registradores, firmware ou `vertex10`. Ele constrói um modelo NNAPI, consulta suporte, solicita compilação para `IDevice/enn` e entrega buffers de operandos ao runtime. O HAL `android.hardware.neuralnetworks-service-enn` atua como fronteira entre a interface Android e o software Samsung ENN.

Dentro da pilha ENN aparecem camadas distintas. `libenn_wrapper` e `libenn_wrapper_system` fazem a ponte de integração. `libenn_model` mantém a representação do grafo. `libenn_engine` e `libenn_engine_lib` participam da preparação e execução. `libenn_user_lib` e `libenn_user.samsung_slsi` conectam o runtime ao driver vendor. Os drivers separados de CPU, GPU e unified indicam que o runtime possui mais de um caminho de execução, portanto o nome `enn` sozinho não revela qual subcomponente executou cada operação.

15.2. Modelo, compilador e formato interno

As strings das bibliotecas mostram `NCP_BINARY`, `NCP Version`, `NCPBuffer`, `NPUCompiler`, `NPUCommon`, `backends NPUCrane`, `NPUDove`, `NPUeagle` e `NPUroot`, além de geradores `runNcpGenerator`, `runCMDQGenerator` e `runISAGenerator`. Isso indica uma cadeia de transformação que parte de um grafo e chega a uma representação NCP/CMDQ específica do acelerador.

Essa evidência é estática. Ela demonstra a presença de conceitos de compilação, command queue e geração de ISA no software instalado, mas não revela a codificação completa dos comandos nem autoriza a montagem de uma fila privada.

15.3. Hardware lógico e blocos internos

O Device Tree nomeia blocos `gnpu0`, `gnpu1`, `snpu0`, `snpu1`, `dnc`, `dsp` e `npumem`. As propriedades `samsung,npucmd-*` incluem ativação e desativação de clocks, DSP, NPU, DNC, STM e caminhos de acesso a registradores. Os nomes `sfrgnpu0`, `sfrgnpu1`, `sfrsnpu0`, `sfrsnpu1`, `sfrdnc`, `sfrdsp0` e `sfrnpumem` aparecem nas tabelas de controle observadas.

O mapa também registra `NPU0` e `NPU1` em nomes de nó, frequência, scheduler e DVFS. Isso indica uma arquitetura com mais de um domínio lógico de processamento, mas não permite concluir que ambos possam ser selecionados individualmente pela API pública.

15.4. Memória, IOMMU e DMA

O nó `npu_exynos` possui referências a `iommu`, `samsung,iommu-group`, `sysmmu,best-fit`, `dma-window`, `dma-coherent`, `samsung,npumem-address`, `samsung,npumem-names` e `samsung,npurmem-address`. A topologia observada liga o dispositivo a um grupo IOMMU e a dois fornecedores SysMMU em sysfs.

Esses elementos mostram que operandos e buffers passam por uma política de endereçamento e tradução própria. O mapa não contém uma prova de que um buffer alocado arbitrariamente por um aplicativo possa ser convertido em memória NPU pelo endpoint direto. A rota pública NNAPI esconde essa operação dentro do HAL e do runtime.

15.5. Frequência, QoS, AFM e energia

O sysfs expõe `qos_freq`, governos de NPU, frequências de `npufreq`, `intfreq` e `miffreq`, tabelas DVFS, limites de atividade, número mínimo de cores, afinidade de interrupção e propriedades AFM como `afm_irp`, `afm_mode`, `afm_onoff`, `afm_restore_msec`, `afm_tdc_threshold` e `afm_tdt`.

Também aparecem nós de throughput para NPU0, NPU1 e NPU agregado. A presença dessa infraestrutura explica por que tempo de execução, frequência e temperatura precisam ser interpretados como parte do estado do sistema. Ela não é uma autorização para escrever em sysfs ou forçar frequências.

15.6. Firmware, wakeup e recuperação

O sistema mantém propriedades de `wakeup` para o dispositivo `npu_exynos`, incluindo contadores de atividade, mudanças, eventos, expirações e tempo ativo. O sysfs também expõe `npu_err_in_dmesg`, `log_level`, `suspend_resume_test` e `version`.

Esses nós representam a integração do acelerador com suspensão, recuperação e diagnóstico do kernel. O mapa confirma a existência dos mecanismos de controle e observabilidade; não confirma o protocolo de recuperação nem a semântica de cada comando.

15.7. Fronteira entre API pública e ABI interna

A API pública é identificável por `IDevice/enn`, operações NNAPI, compilação e execução. A ABI interna aparece nos binários vendor, no endpoint `vertex10`, nos símbolos NCP/CMDQ e nas propriedades Device Tree. Entre as duas existe uma fronteira deliberada de HAL, namespaces, SELinux, permissões de dispositivo e formatos proprietários.

O fato de o caminho NNAPI funcionar sem root e o caminho `vertex10` falhar para UID comum é uma evidência dessa separação. O XOP deve documentar essa fronteira, não tratá-la como uma simples ausência de comando.

16. Proveniência e classificação

Fonte: pacote `Exynos_NPU.zip`, recebido em 18/09/2026. SHA-256 do pacote:
`873a0f8d3ff4d8e9a4168aafab0349f5becfa539cfc9b03d8a7033e3acb08af9`.

Os itens 10 e 12 constituem a evidência de execução mais forte. Os itens 13b, 15, 16 e 17 constituem evidência de limites ou investigação incompleta. Os itens 09b e 09c precisam ser mantidos como falhas de instrumentação, não como prova de comportamento do hardware.

Logs crus, comandos de coleta, binários compilados, bibliotecas vendor, firmware, modelos proprietários, dumps e informações pessoais não fazem parte da distribuição pública.